

Linear Algebra Exercises / 線形代数の演習

Topic / トピック: Vectors and Matrices / ベクトルと行列

Instructions / 指示:

- Complete each exercise by writing your code in the cell marked `## Your code here`
- 各演習について、`## Your code here` と書かれたセルにコードを書いて完成させてください
- Run the test cell after each exercise to check your answer
- 各演習の後にテストセルを実行して、答えを確認してください

```
# Run this cell first / 最初にこのセルを実行してください
import numpy as np
import matplotlib.pyplot as plt
```

Exercise 1: Vector Operations / ベクトル演算

Given two vectors $\vec{u} = [1, 2, 3]$ and $\vec{v} = [4, 5, 6]$, compute:

2つのベクトル $\vec{u} = [1, 2, 3]$ と $\vec{v} = [4, 5, 6]$ について、以下を計算してください:

1. The sum $\vec{u} + \vec{v}$ / ベクトルの和
2. The dot product $\vec{u} \cdot \vec{v}$ / 内積
3. The magnitude (norm) of $\vec{u}$ / $\vec{u}$ の大きさ (ノルム)
4. The angle between them in degrees / 2つのベクトル間の角度 (度)

```
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])

## Your code here / ここにコードを書いてください
vector_sum = u + v      # u + v
dot_product = np.dot(u, v) # u · v
norm_u = np.linalg.norm(u) # ||u||
norm_v = np.linalg.norm(v)
cos_theta = dot_product / (norm_u * norm_v)
angle_radians = np.arccos(cos_theta)
angle_degrees = np.degrees(angle_radians) # angle between u and v in degrees

print('Sum / 和:', vector_sum)
print('Dot product / 内積:', dot_product)
print('Norm of u / u のノルム:', norm_u)
print('Angle / 角度:', angle_degrees, 'degrees')

Sum / 和: [5 7 9]
Dot product / 内積: 32
Norm of u / u のノルム: 3.7416573867739413
Angle / 角度: 12.933154491899135 degrees
```

Exercise 2: Matrix Transpose / 行列の転置

Write a function `my_transpose(A)` that returns the transpose of matrix A **without using** `np.transpose` or `.T`.

`np.transpose` や `.T` を**使わずに**、行列 A の転置を返す関数 `my_transpose(A)` を書いてください。

Hint / ヒント: $A^T[i][j] = A[j][i]$

```
def my_transpose(A):
    ## Your code here / ここにコードを書いてください
    rows = len(A)
    cols = len(A[0])

    # Create a new matrix with swapped dimensions
    transposed_A = [[0 for _ in range(rows)] for _ in range(cols)]

    # Populate the new matrix by swapping indices
    for i in range(rows):
        for j in range(cols):
            transposed_A[j][i] = A[i][j]
    return transposed_A

# Test / テスト
A = [[1, 2, 3],
     [4, 5, 6]]
print('Original / 元の行列:', A)
print('Transpose / 転置:', my_transpose(A))
# Expected / 期待される結果: [[1, 4], [2, 5], [3, 6]]
```

```
Original / 元の行列: [[1, 2, 3], [4, 5, 6]]
Transpose / 転置: [[1, 4], [2, 5], [3, 6]]
```

Exercise 3: Matrix Multiplication / 行列の積

Compute $C = AB$ for the following matrices using NumPy.

NumPyを使って、以下の行列について $C = AB$ を計算してください。

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Then verify that $AB \neq BA$ in general.

そして、一般に $AB \neq BA$ であることを確認してください。

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

## Your code here / ここにコードを書いてください
AB = A @ B
BA = B @ A

print('AB =\n', AB)
print('BA =\n', BA)
print('AB == BA?', np.array_equal(AB, BA))
```

```
AB =
[[19 22]
 [43 50]]
BA =
[[23 34]
 [31 46]]
AB == BA? False
```

Exercise 4: Determinant and Inverse / 行列式と逆行列

For the matrix M below:

以下の行列 M について：

$$M = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 0 & 2 \\ 4 & 1 & 5 \end{bmatrix}$$

1. Compute the determinant / 行列式を計算してください
2. Determine whether it is invertible / 逆行列が存在するか判定してください
3. If invertible, compute the inverse / 逆行列が存在する場合は、それを計算してください
4. Verify that $M \cdot M^{-1} = I$ / $M \cdot M^{-1} = I$ を確認してください

```
M = np.array([[2, 1, 3],
              [1, 0, 2],
              [4, 1, 5]])

## Your code here / ここにコードを書いてください
det_M = np.linalg.det(M)
is_invertible = det_M != 0
```

```
M_inv = None # Set to None if not invertible / 逆行列がない場合は None

if is_invertible:
    M_inv = np.linalg.inv(M)

print('Determinant / 行列式:', det_M)
print('Invertible / 可逆?:', is_invertible)
if M_inv is not None:
    print('Inverse / 逆行列:\n', M_inv)
    print('M @ M_inv =\n', np.round(M @ M_inv, 5))
```

```
Determinant / 行列式: 2.0
Invertible / 可逆?: True
Inverse / 逆行列:
[[-1. -1.  1.]
 [ 1.5 -1. -0.5]
 [ 0.5  1. -0.5]]
M @ M_inv =
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Exercise 5: Solving Linear Systems / 連立一次方程式を解く

Solve the following system of equations using NumPy:

NumPyを使って、以下の連立方程式を解いてください:

$$\begin{cases} 2x + y - z = 8 \\ -3x - y + 2z = -11 \\ -2x + y + 2z = -3 \end{cases}$$

Hint / ヒント: Use `np.linalg.solve(A, b)`

```
## Your code here / ここにコードを書いてください
A = np.array([[2, 1, -1],
              [-3, -1, 2],
              [-2, 1, 2]]) # coefficient matrix / 係数行列
b = np.array([8, -11, -3]) # right-hand side vector / 右辺ベクトル
solution = np.linalg.solve(A, b) # [x, y, z]

print('Solution / 解:', solution)
print('x =', solution[0] if solution is not None else None)
print('y =', solution[1] if solution is not None else None)
print('z =', solution[2] if solution is not None else None)
```

```
Solution / 解: [ 2.  3. -1.]
x = 2.0000000000000004
y = 2.9999999999999996
z = -0.9999999999999994
```

Exercise 6: Linear Transformation / 線形変換

A linear transformation $T: \mathbb{R}^3 \rightarrow \mathbb{R}^2$ is defined by:

線形変換 $T: \mathbb{R}^3 \rightarrow \mathbb{R}^2$ を以下のように定義します:

$$T\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} x_1 - 2x_2 + x_3 \\ 3x_1 + x_2 - x_3 \end{bmatrix}$$

- Find the standard matrix representation of T / T の標準行列表示を求めてください
- Determine if T is **onto** (surjective) / T が**全射**であるか判定してください
- Determine if T is **one-to-one** (injective) / T が**単射**であるか判定してください
- Determine if T is **invertible** / T が**可逆**であるか判定してください

```
## Your code here / ここにコードを書いてください
T = np.array([[1, -2, 1],
              [3, 1, -1]]) # standard matrix / 標準行列

rank = np.linalg.matrix_rank(T)

# T: R^3 -> R^2
# Dimension of codomain (R^2) is 2
is_onto = rank == 2 # T is onto if rank(T) == dim(codomain)

# Dimension of domain (R^3) is 3
is_one_to_one = rank == 3 # T is one-to-one if rank(T) == dim(domain)
```

```
# T is invertible if it's both one-to-one and onto, and the matrix is square.
# Since the matrix is 2x3 (not square), it cannot be invertible.
is_invertible = False

print('Standard matrix / 標準行列:¥n', T)
print('Rank / 階数:', rank)
print('Onto / 全射:', is_onto)
print('One-to-one / 単射:', is_one_to_one)
print('Invertible / 可逆:', is_invertible)
```

```
Standard matrix / 標準行列:
[[ 1 -2  1]
 [ 3  1 -1]]
Rank / 階数: 2
Onto / 全射: True
One-to-one / 単射: False
Invertible / 可逆: False
```

✓ Exercise 7: Eigenvalues and Eigenvectors / 固有値と固有ベクトル

For the matrix:

次の行列について：

$$A = \begin{bmatrix} 4 & -2 \\ 1 & 1 \end{bmatrix}$$

1. Find all eigenvalues / すべての固有値を求めてください
2. Find the corresponding eigenvectors / 対応する固有ベクトルを求めてください
3. Verify that $A\vec{v} = \lambda\vec{v}$ for the first eigenpair / 最初の固有値・固有ベクトルの組について $A\vec{v} = \lambda\vec{v}$ を確認してください

Hint / ヒント: Use `np.linalg.eig(A)`

```
A = np.array([[4, -2],
              [1,  1]])

## Your code here / ここにコードを書いてください
eigenvalues, eigenvectors = np.linalg.eig(A)

print('Eigenvalues / 固有値:', eigenvalues)
print('Eigenvectors / 固有ベクトル (as columns / 列として):', eigenvectors)

# Verification for the first pair / 最初の組の検証
if eigenvalues is not None and eigenvectors is not None:
    v1 = eigenvectors[:, 0]
    lam1 = eigenvalues[0]
    print('¥nA @ v1 =', A @ v1)
    print('lambda1 * v1 =', lam1 * v1)
```

```
Eigenvalues / 固有値: [3.  2.]
Eigenvectors / 固有ベクトル (as columns / 列として): [[0.89442719 0.70710678]
 [0.4472136  0.70710678]]

A @ v1 = [2.68328157 1.34164079]
lambda1 * v1 = [2.68328157 1.34164079]
```

✓ Exercise 8: Visualizing a Linear Transformation / 線形変換の可視化

The matrix below rotates a 2D vector by 45 degrees counterclockwise.

以下の行列は、2次元ベクトルを反時計回りに45度回転させます。

$$R = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad \theta = 45^\circ$$

Apply R to the vector $\vec{v} = [2, 1]$ and plot both the original and rotated vectors.

ベクトル $\vec{v} = [2, 1]$ に R を適用し、元のベクトルと回転後のベクトルを両方プロットしてください。

```

theta = np.pi / 4 # 45 degrees
v = np.array([2, 1])

## Your code here / ここにコードを書いてください
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]]) # rotation matrix / 回転行列
v_rotated = R @ v

print('Rotation matrix / 回転行列:¥n', R)
print('Original / 元のベクトル:', v)
print('Rotated / 回転後:', v_rotated)

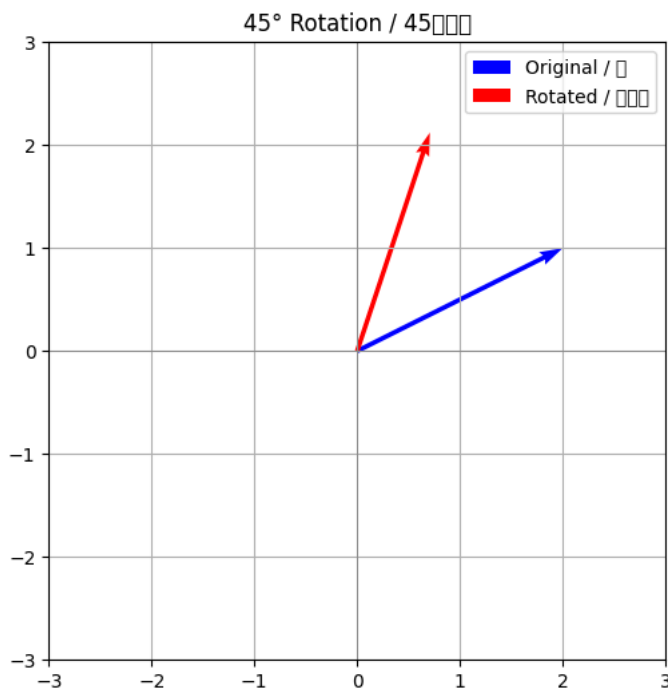
# Plot / プロット
if R is not None and v_rotated is not None:
    plt.figure(figsize=(6, 6))
    plt.quiver(0, 0, v[0], v[1], angles='xy', scale_units='xy', scale=1, color='blue', label='Original / 元')
    plt.quiver(0, 0, v_rotated[0], v_rotated[1], angles='xy', scale_units='xy', scale=1, color='red', label='Rotated / 回転後')
    plt.xlim(-3, 3)
    plt.ylim(-3, 3)
    plt.axhline(0, color='gray', linewidth=0.5)
    plt.axvline(0, color='gray', linewidth=0.5)
    plt.grid(True)
    plt.legend()
    plt.title('45° Rotation / 45度回転')
    plt.gca().set_aspect('equal')
    plt.show()

```

```

Rotation matrix / 回転行列:
[[ 0.70710678 -0.70710678]
 [ 0.70710678  0.70710678]]
Original / 元のベクトル: [2 1]
Rotated / 回転後: [0.70710678 2.12132034]
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning:
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning:
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning:
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning:
  fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning:
  fig.canvas.print_figure(bytes_io, **kw)

```



✓ Challenge Exercise / チャレンジ問題

Write a function `is_linearly_independent(vectors)` that takes a list of vectors and returns `True` if they are linearly independent, `False` otherwise.

ベクトルのリストを受け取り、それらが**一次独立**であれば `True`、そうでなければ `False` を返す関数

`is_linearly_independent(vectors)` を書いてください。

Hint / ヒント: Vectors are linearly independent if and only if the rank of the matrix formed by them equals the number of vectors.

ベクトルが一次独立であるための必要十分条件は、それらを並べた行列の階数がベクトルの数に等しいことです。

```
def is_linearly_independent(vectors):  
    ## Your code here / ここにコードを書いてください  
    if not vectors: # Handle empty list of vectors  
        return True  
  
    # Convert list of vectors to a NumPy array (matrix)  
    matrix = np.array(vectors)  
  
    # Number of vectors is the number of rows in the matrix  
    num_vectors = matrix.shape[0]  
  
    # Calculate the rank of the matrix  
    rank = np.linalg.matrix_rank(matrix)  
  
    # Vectors are linearly independent if the rank equals the number of vectors  
    return rank == num_vectors  
  
# Tests / テスト  
v1 = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]  
v2 = [[1, 2, 3], [2, 4, 6], [1, 0, 0]]  
v3 = [[1, 1], [2, 3], [4, 5]]  
  
print('Test 1 (should be True / Trueになるはず):', is_linearly_independent(v1))  
print('Test 2 (should be False / Falseになるはず):', is_linearly_independent(v2))  
print('Test 3 (should be False / Falseになるはず):', is_linearly_independent(v3))
```

```
Test 1 (should be True / Trueになるはず): True  
Test 2 (should be False / Falseになるはず): False  
Test 3 (should be False / Falseになるはず): False
```

Submission / 提出

After completing all exercises, save your notebook and submit it